



Titanic Survival

Un'API Flask per prevedere la sopravvivenza dei passeggeri del Titanic utilizzando algoritmi di Machine Learning supervisionato: **Regressione Logistica** e **XGBoost**.

Disponibile su: <https://github.com/gianlucameni/titanic-survival>

A cura di: Gianluca Meni e Samuele Ghezzi

Indice

Presentazione del Dataset

Esplorazione delle feature, distribuzioni e relazioni tra variabili

1

2

Struttura del Progetto

Organizzazione dei package e dei file principali

3

Pipeline di Lavoro

Flusso completo, dal dato grezzo alla previsione

4

Encoding & Preprocessing

Gestione valori mancanti, outlier, encoding e feature engineering

5

Modelli ML

Regressione Logistica e XGBoost a confronto

6

Tuning degli Iperparametri

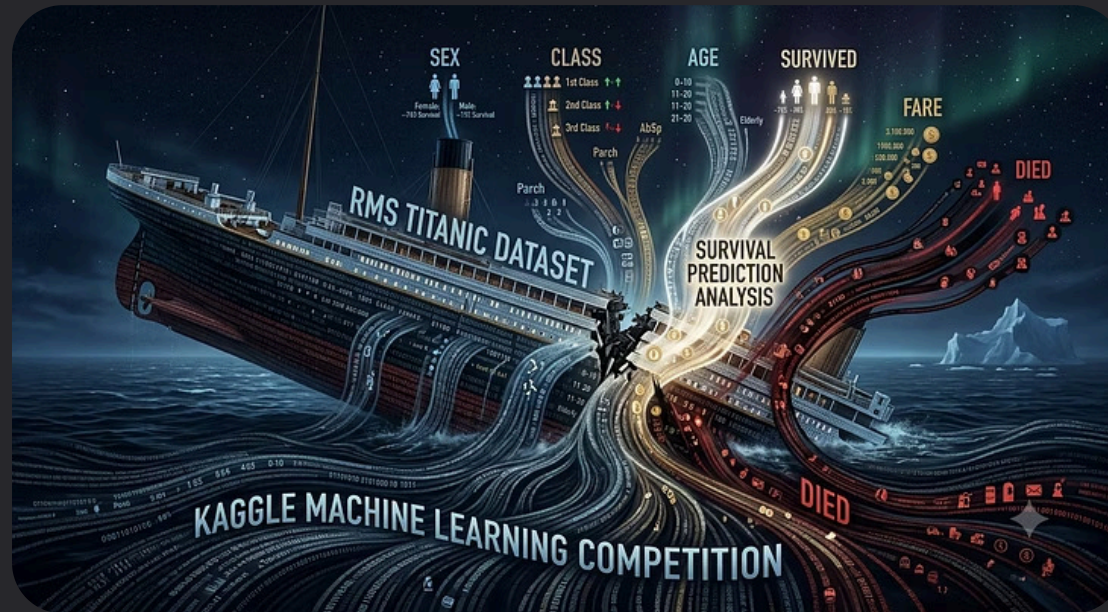
Ottimizzazione dei modelli con GridSearchCV per migliorare le performance

7

Risultati & Sviluppi Futuri

Metriche di valutazione e possibili sviluppi del progetto

Presentazione del Dataset



Il **dataset** Titanic di Kaggle contiene informazioni su **891 passeggeri** con le seguenti features:

Identificativi

PassengerId, Name,
Ticket, Cabin

Demografici

Sex, Age, SibSp,
Parch

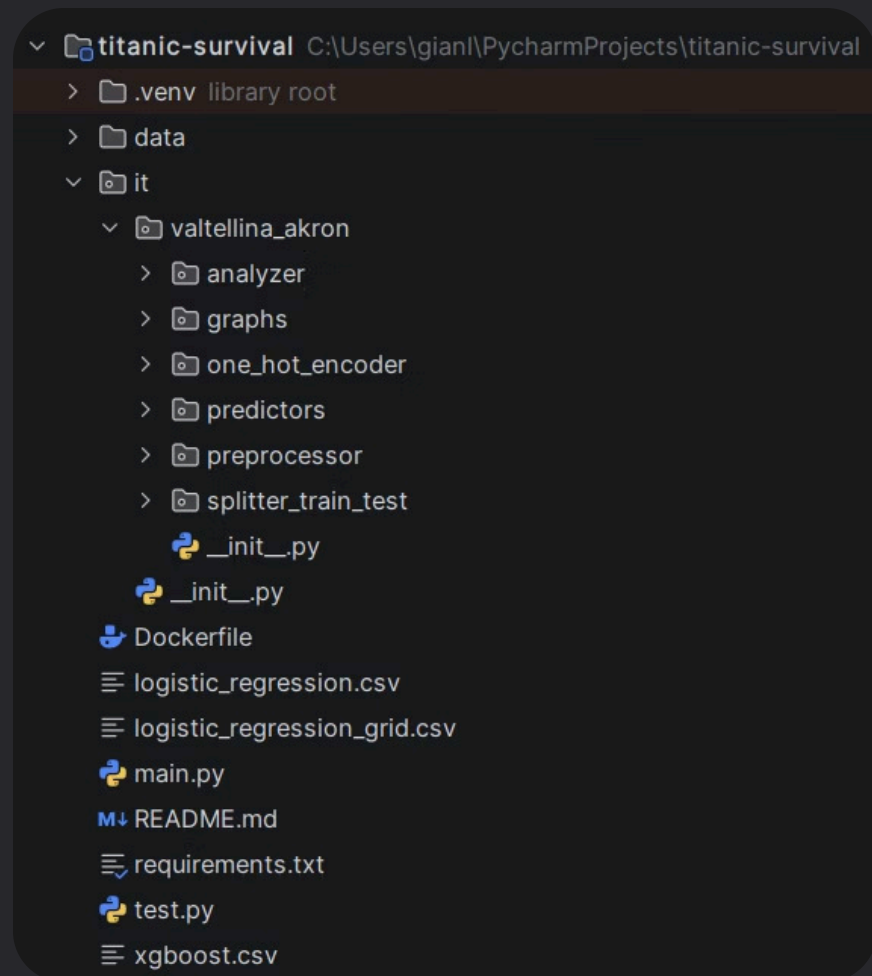
Viaggio

Pclass, Embarked,
Fare

Target

Survived
(0 = No, 1 = Sì)

Struttura del Progetto



main.py: Script principale per avviare il flusso di analisi, training e valutazione

test.py: Script dedicato ai test e alla verifica del comportamento dei componenti del progetto.

Dockerfile: Definisce l'ambiente Docker per rendere l'esecuzione del progetto riproducibile.

requirements.txt: Elenca tutte le dipendenze necessarie per installare ed eseguire il progetto.

README.md: Fornisce una panoramica del progetto e setup ambiente locale / Docker

it/valtellina_akron: contiene tutti i moduli per il funzionamento del progetto:

/analyzer

Modulo per l'analisi e la preparazione dei dati.

/graphs

Modulo per la creazione di grafici.

/one_hot_encoder

Gestisce la codifica delle feature categoriche tramite OHE.

/predictors

Contiene la definizione, il fitting e la valutazione dei modelli.

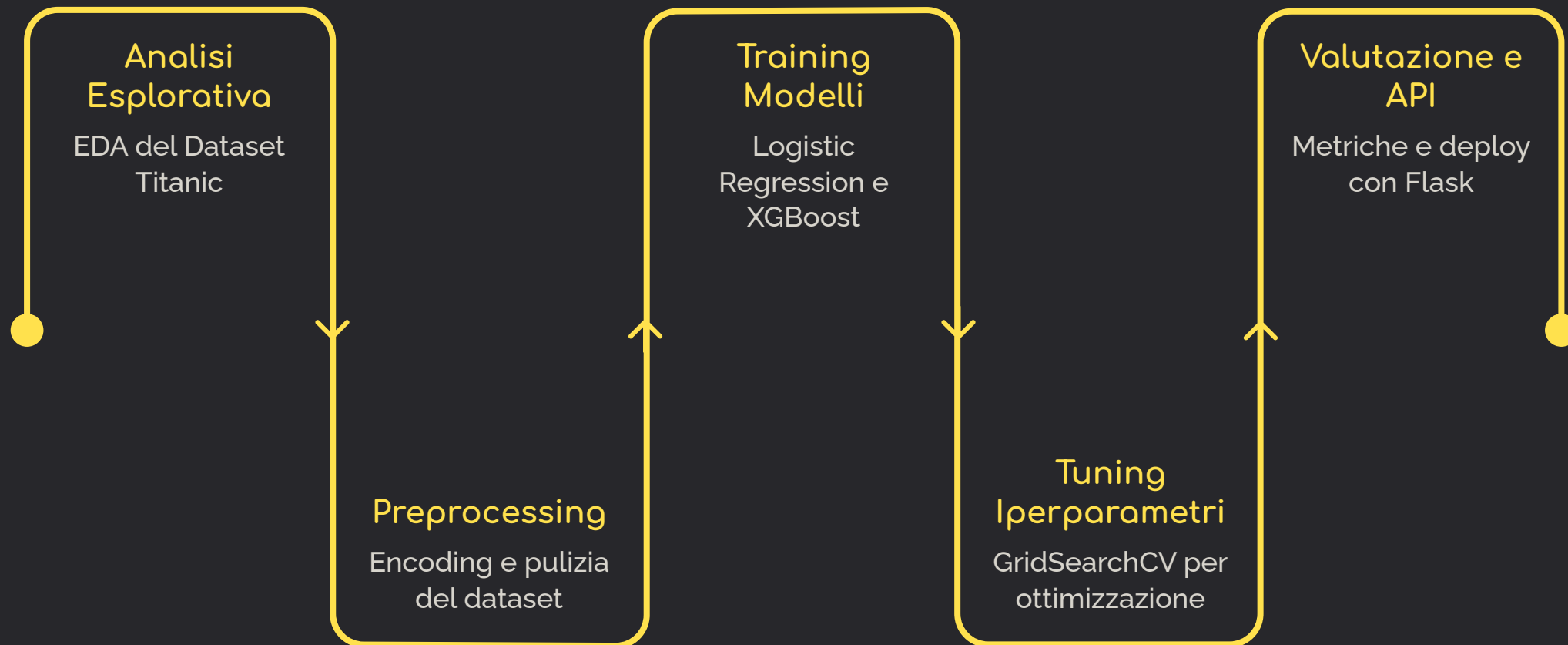
/preprocessor

Applica Robust Scaler per normalizzare i dati.

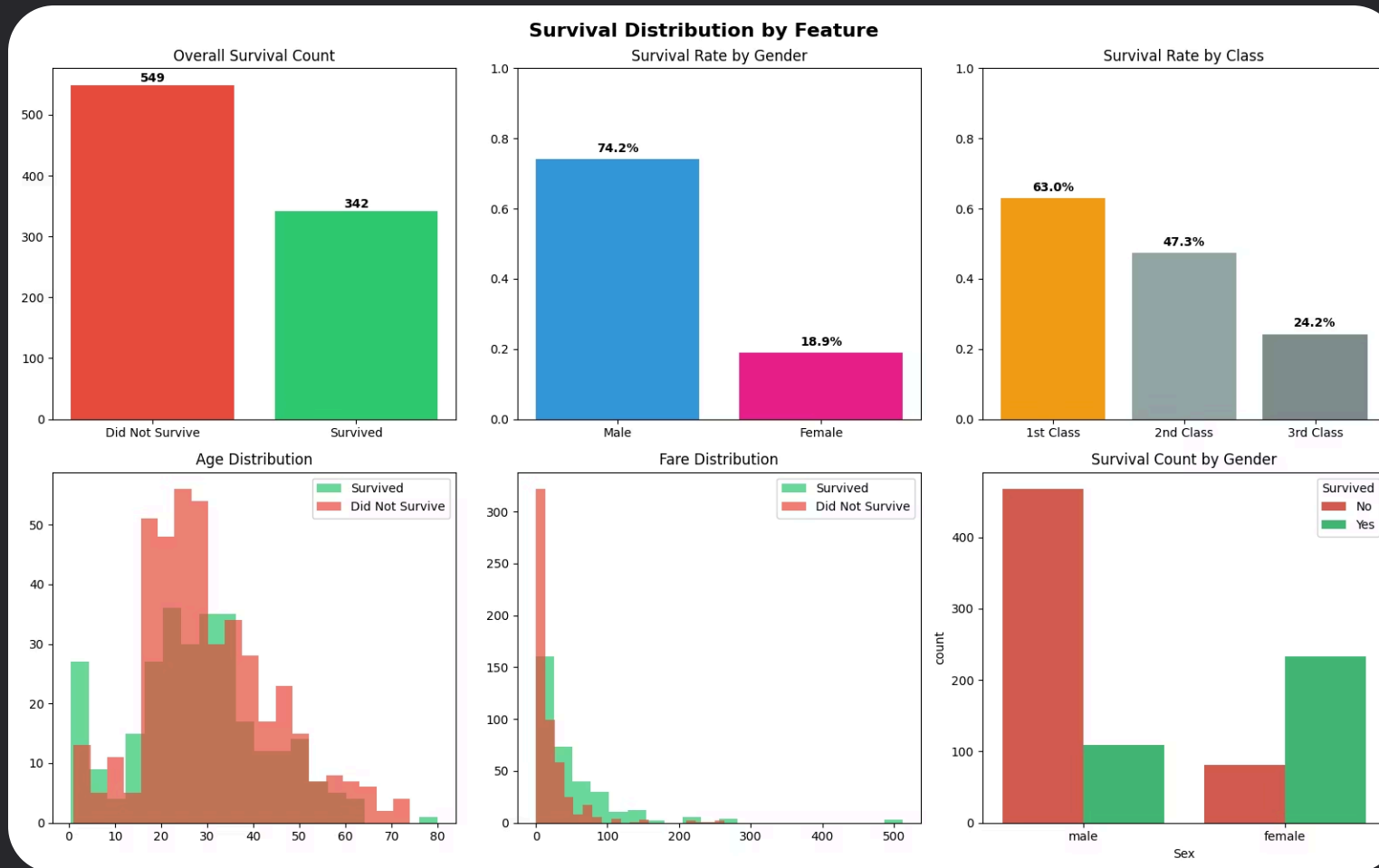
/splitter_train_test

Si occupa della suddivisione del dataset in training set e test set.

Pipeline di Lavoro



Analisi Esplorativa



L'EDA mostra dei pattern legati alla sopravvivenza:

- Il dataset risulta **sbilanciato**, con il 60.6% di non sopravvissuti.
- Le donne hanno avuto tassi di sopravvivenza più alti rispetto agli uomini.
- I passeggeri di prima classe avevano probabilità di sopravvivenza superiori rispetto a seconda e terza classe.
- Tariffe più alte risultano associate a una maggiore probabilità di sopravvivenza.

Encoding & Preprocessing

Gestione Colonne

Rimozione di PassengerId, Ticket, Cabin e Embarked e aggiunta di Title e Family_Size

Missing Values

Imputazione con mediana per Age e Fare una volta verificata la distribuzione non normale

Outliers & Duplicati

Controllo e gestione di possibili outliers e duplicati presenti nel dataset

One-Hot e Label Encoding

Conversione delle variabili categoriche (Sex, Title) in formato numerico compatibile con i modelli ML

Feature Engineering

Campo Title

Il campo **Name** contiene il titolo, che cattura informazioni su **genere, età e classe sociale** in un'unica variabile. Il titolo onorifico (Mr, Mrs, Miss, Master, Dr...) viene estratto come nuova feature categorica.

❗ Esempio: "*Braund, Mr. Owen Harris*" → titolo **Mr**, indicatore di genere e status sociale.

Dopo l'estrazione, la colonna **Name** viene eliminata dal dataset e la nuova colonna **Title** convertita tramite One Hot Encoding.

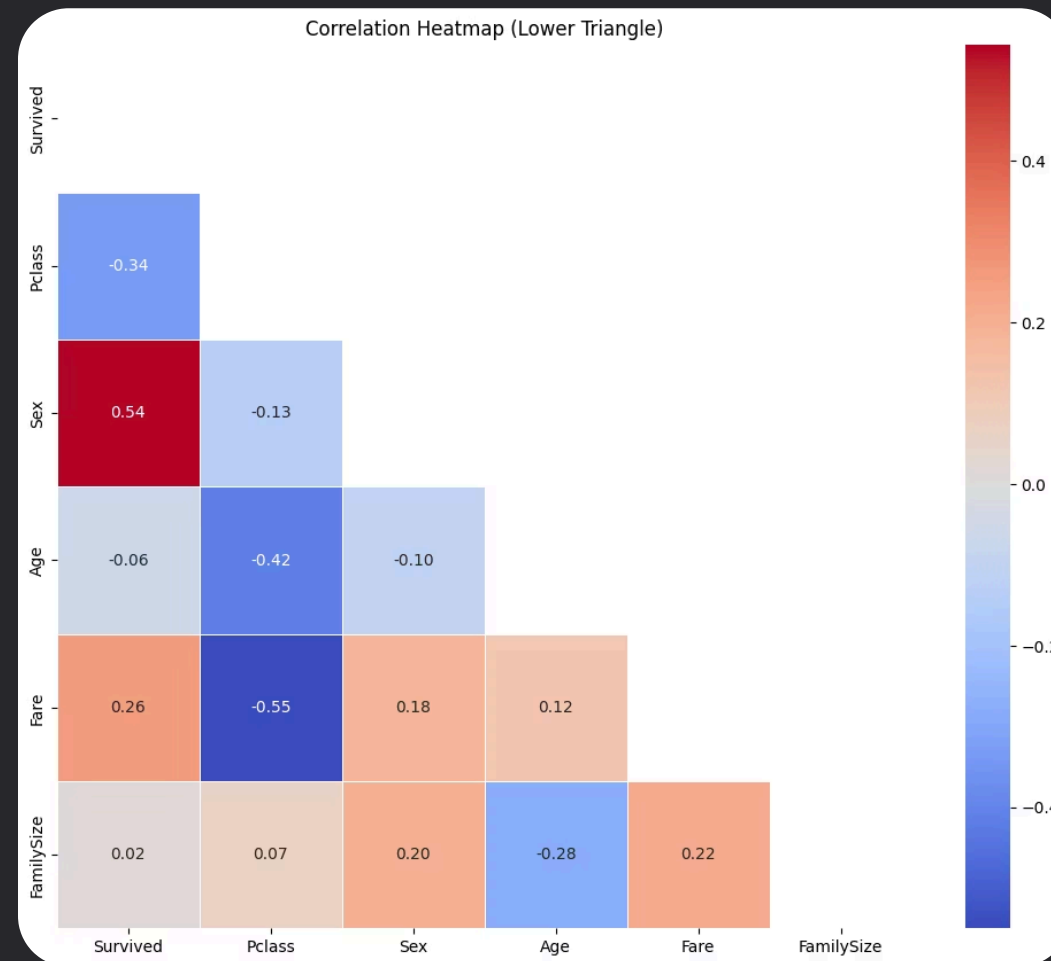
Campo FamilySize

Le feature **SibSp** (numero di fratelli e sposi) e **Parch** (numero di genitori e figli) vengono combinate in una unica feature che rappresenta la dimensione della famiglia, riducendo la dimensione del dataset ma fornendo la stessa informazione.

Analisi della Correlazione

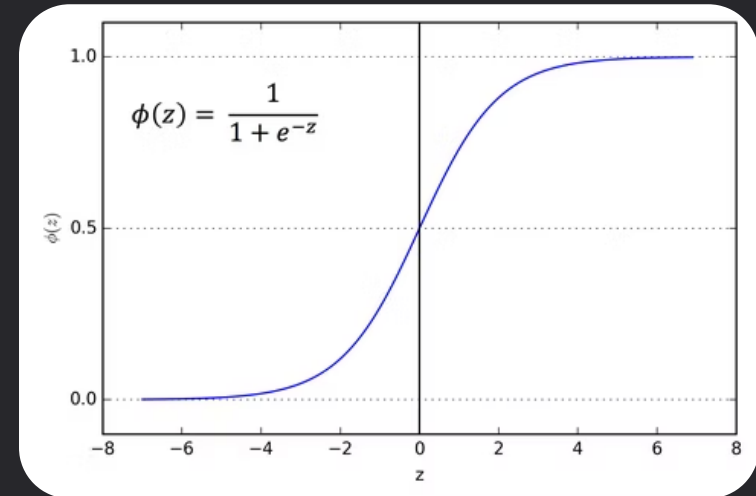
L'analisi della correlazione permette di comprendere quali feature influenzano maggiormente la probabilità di sopravvivenza.

- Emerge che il sesso (**Sex**) è la feature che impatta di più sul target.
- Emerge una correlazione negativa tra la Classe (**PClass**) e il costo del biglietto (**Fare**).



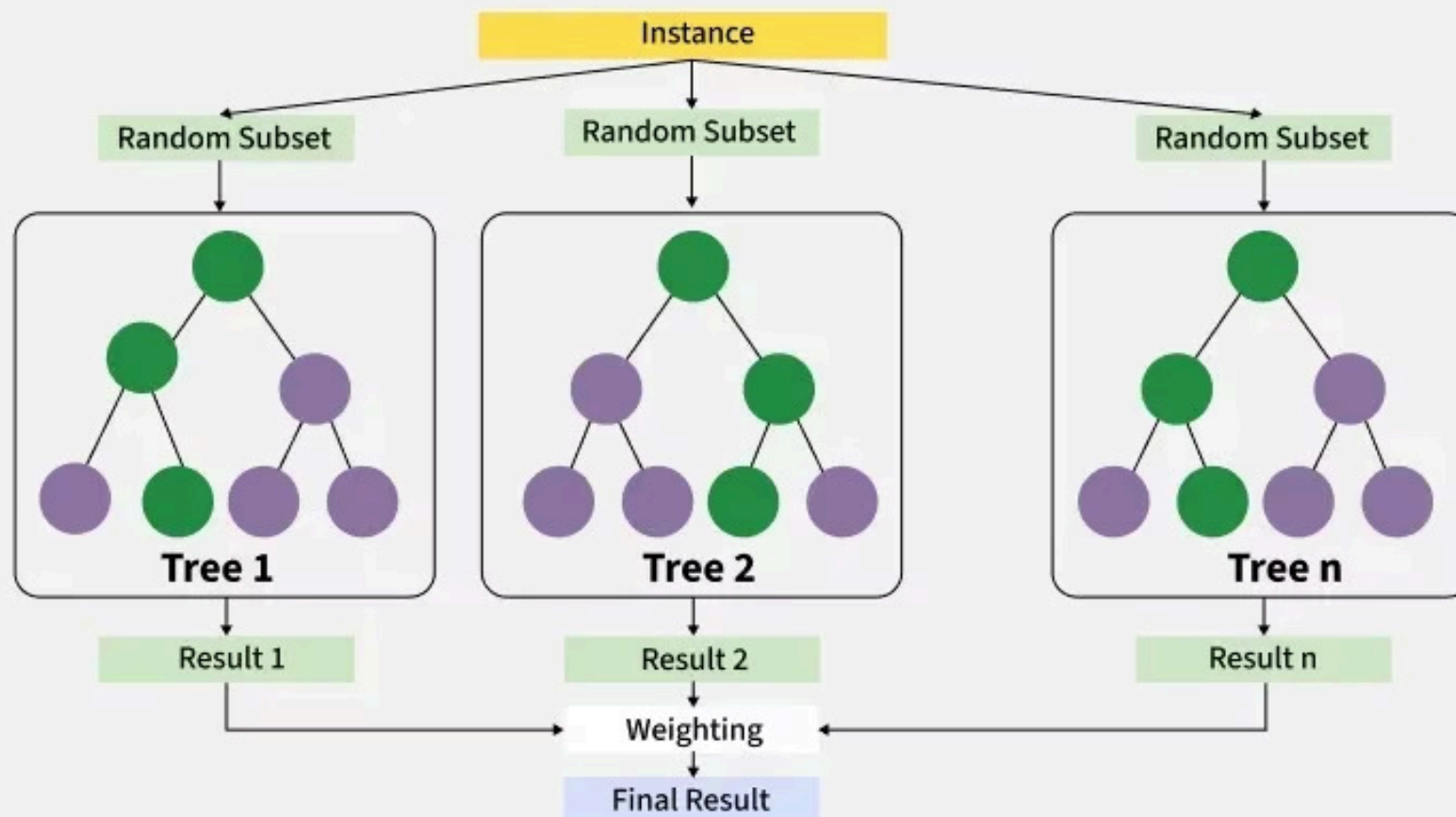
Regressione Logistica

- Usa la funzione **sigmoide** per trasformare l'output in una probabilità compresa tra 0 e 1
- restituisce la probabilità di appartenenza alla **classe positiva**
- dello per la **classificazione binaria**
- obiettivo: predire probabilità in modo interpretabile



XGBoost

- Algoritmo di **gradient boosting** basato su alberi di decisione.
- Combina modelli deboli in **sequenza**, correggendo gli errori del precedente.
- Ogni nuovo albero cerca di **correggere gli errori** (residui) accumulati dagli alberi precedenti.
- Il modello finale è la somma dei contributi di tutti gli alberi.
- Il contributo di ogni albero è regolato dal **learning rate**: nuova previsione = previsione precedente + learning_rate × correzione.
- Ad ogni iterazione il modello riduce progressivamente l'errore e migliora le prestazioni.



Metriche di Valutazione

✓ ACCURACY

La percentuale di previsioni corrette (veri positivi e veri negativi) sul totale delle previsioni effettuate. Indica l'accuratezza complessiva del modello.

🔍 PRECISION

La proporzione di veri positivi tra tutte le istanze che il modello ha classificato come positive. Importante per ridurre i falsi allarmi.

🕒 RECALL

La proporzione di veri positivi che sono stati correttamente identificati dal modello tra tutte le istanze positive reali. È fondamentale quando i **falsi negativi** sono rischiosi.

📊 F1-SCORE

La media armonica di Precision e Recall. Fornisce un equilibrio tra le due metriche, ed è particolarmente utile con dataset sbilanciati.

Risultati dei Modelli

Le performance dei modelli vengono valutate tramite metriche standard per la classificazione binaria. La tabella seguente confronta Logistic Regression e XGBoost sui risultati finali del test.

Metrica	Logistic Regression	XGBoost
Accuracy	0.8436	0.8156
Precision	0.8254	0.7647
Recall	0.7536	0.7536
F1-Score	0.7879	0.7591

❏ Logistic Regression ottiene risultati migliori in Accuracy, Precision e F1-Score, mentre Recall è identico per entrambi i modelli.

Tuning degli Iperparametri

Ottimizzazione con GridSearch

Per massimizzare le performance del modello di Regressione Logistica, è stata implementata una fase di tuning degli iperparametri utilizzando **GridSearchCV**, tecnica che esplora sistematicamente diverse combinazioni di iperparametri per trovare il set ottimale che minimizza l'errore sul validation set.

 Il processo ha permesso di identificare gli iperparametri più efficaci per la Regressione Logistica, migliorando i risultati delle metriche,

Iperparametri Ottimali

C

100 (Forza della **regolarizzazione**: se troppo grande c'è rischio overfitting)

penalty

L1 (Regolarizzazione Lasso)

solver

saga (aggiorna pesi del gradiente, supportando l1, l2 ed elasticnet)

Confronto Metriche: Prima e Dopo Tuning

Metric	Before Tuning	After Tuning
Accuracy	0.8436	0.8659
Precision	0.8254	0.8462
Recall	0.7536	0.7971
F1-Score	0.7879	0.8209

- ✔ L'ottimizzazione con GridSearch ha portato a un miglioramento significativo in tutte le metriche, dimostrando l'efficacia del tuning degli iperparametri.

Sviluppi Futuri



Feature Engineering Avanzato

Nuove combinazioni di feature per una migliore analisi

Aggiunta di più modelli ML

Implementazione di ulteriori modelli di predizione per ricercare metriche migliori

Aggiornamento API Flask

Nuovi endpoint REST per ampliare la disponibilità di grafici e modelli